Chapter 1

Deep Learning

This chapter introduces the fundamental concepts of Deep Learning (DL), providing the necessary foundation for the ideas developed throughout the rest of this manuscript. This chapter can be seen as a synthesis of the reference work Deep Learning [Goodfellow et al., 2016]. Its structure is as follows:

Section 1.1: An overview of the historical developments that led to the emergence of the field.

Section 1.2: A formalization of the objective underlying DL algorithms.

Section 1.3: An examination of the structure of artificial Neural Network (NN).

Section 1.4: An explanation of how the learning process shapes NN.

Section 1.5: A discussion of how such models are evaluated.

1.1 Historical Developments

Figure 1.1 provides a timeline of the key milestones that shaped deep learning, from the earliest mathematical model of artificial neurons to the rise of modern transformer architectures.

DL and NN emerged from an ambition to study the expressive capabilities of organic brain. They can be seen as a particularly striking example of biomimicry, as they draw inspiration from living systems to address scientific challenges. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [McCulloch and Pitts, 1943]. This work established the key idea that networks of simple units can give rise to complex computations. The focus was on representational power, not on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications. A major step toward operationalizing the idea of networks with the perceptron algorithm in 1958 [Rosenblatt, 1958]. The perceptron was designed to adjusting its parameters based on input–output pairs in order to approximate a function. However, the perceptron's fundamental limitations were exposed in 1969, revealing that the algorithm could not learn functions as simple as

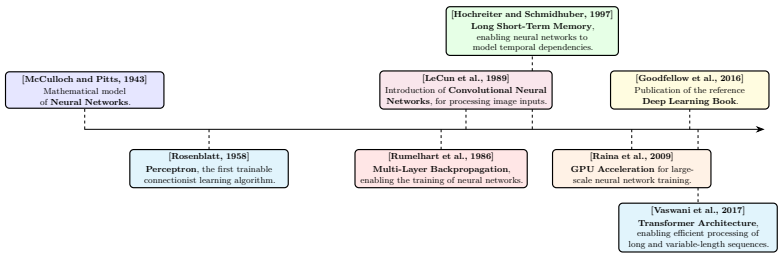


Figure 1.1: Evolution of deep learning.

the exclusive-or [Minsky and Papert, 1969]. To overcome this limitation, researchers in 1986 proposed using gradient descent optimization to train deep NNs [Rumelhart et al., 1986].

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation theorem shows that NN can represent a wide class of functions [Hornik et al., 1989], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [Hilbert and López, 2011]. At the same time, progress in hardware, particularly the use of graphics processing units, greatly increased computational capacity and made it feasible to train large-scale models [Krizhevsky et al., 2012].

These societal changes paved the way for the rise of DL as we know it today. DL has achieved remarkable success across a wide range of domains, including computer vision [Krizhevsky et al., 2012], complex games [Silver et al., 2017], content recommendation [Covington et al., 2016], bio-chemistry [Jumper et al., 2021], as well as the now essential digital assistants based on large language models [Brown et al., 2020]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once considered automatable.

1.2 Objective

Now that we have outlined the main developments that led to the rise of DL, we need to formalize what we are actually trying to achieve when performing DL. The range of problems that can be addressed with DL is extremely broad. To ensure clarity, we will consider the supervised learning setting, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as

an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. It gradually becomes capable of capturing the relationship between inputs and outputs through successive updates during training. For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

> "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E." [Mitchell, 1997]

When this formulation is adapted to DL, we obtain:

- The computer program corresponds to the learning algorithm, which is responsible for adjusting the network's parameters.
- The task T consists of learning a NN that provides a good approximation of the target function.
- The performance measure P corresponds to a function used to evaluate the quality of this approximation.
- The experience E takes the form of a dataset composed of input–output pairs derived from the function to be approximated.

We now introduce the mathematical formalism used in this chapter in order to remove any ambiguity in the developments that follow. The function we aim to approximate is denoted by f. As with any function, it defines a mapping from an input to an output: we write $x \in \mathcal{X}$ for an input and $y \in \mathcal{Y}$ for the corresponding output. The function f is therefore characterized by an input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \to \mathcal{Y}$. For simplicity of exposition and notation, we will assume in what follows that both $\mathcal{X}$ and $\mathcal{Y}$ are vector spaces, while keeping in mind that this framework can be readily generalized to more settings. Throughout this manuscript, the notation $\hat{\cdot}$ will be used to indicate that a given object is an approximation of another. Since the goal of the NN we train is to approximate f, we denote this approximation by $\hat{f} : \mathcal{X} \to \mathcal{Y}$. Given an input $x \in \mathcal{X}$, the output of the approximating function $\hat{f}(x)$ is denoted by $\hat{y}$.

It is important to emphasize that the target function f is generally not known explicitly. If it were, there would be no need to learn an approximation, since the true function would already provide the exact mapping. In practice, we only have access to a finite set of observations generated by f, organized in what is called a dataset. To distinguish between the different samples, we introduce an index i, allowing us to define input–output pairs of the form $(x^{(i)}, y^{(i)})$. This leads to the following formulation for a dataset $\mathcal{D}$ of size N, where N denotes the number of available examples:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}.$$

Now that we have made explicit how the target function f is represented, we can ask what it means for a function $\hat{f}$ to be a good approximation of this

function. It means that, for a given input $x^{(i)}$, the NN output $\hat{y}^{(i)}$ is close to the true output $y^{(i)}$. To quantify the quality of this approximation, we introduce a function $\mathcal{L}$ that measures the discrepancy between predictions and target values. In DL, this function called the loss function plays a central role as it guides the learning process. In regression settings, a common choice is the mean squared error, defined as:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2.$$

Thanks to this loss, it is now possible to rigorously define the performance measure P. This performance measure corresponds to the average loss over the dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}(x^{(i)}), y^{(i)}).$$

This formulation captures the mathematical core of the learning objective. We seek a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$. As you can see, we are working in a very general setting, where the target function f may take many different forms. Consequently, the network $\hat{f}$ must be expressive enough to approximate a wide variety of such functions. This requirement motivates the choice of NN architectures, which are specifically designed to represent highly diverse functional mappings.

1.3 Neural Network Structure

NN are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation throughout this chapter, we focus on the Multi-Layer Perceptron (MLP) architecture. Although more specialized architectures exist, the MLP forms a fundamental building block of modern DL. Indeed, most of the essential mechanisms of DL are present in it, making it a particularly suitable framework for introducing key concepts.

1.3.1 Neurons

The artificial neuron is the basic building block of NN. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons and produces an output that is transmitted to subsequent neurons units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,

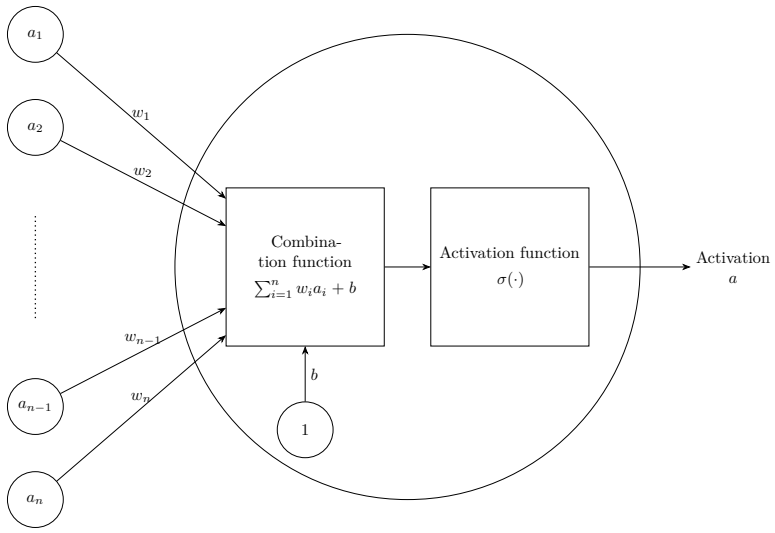


Figure 1.2: Artificiale neurone.

- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,
- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(a_1, \ldots, a_n)$, its activation a is given by:

$$a = \sigma\left(\sum_{i=1}^{n} w_i a_i + b\right),$$

where:

- w_i is the weight associated with the i-th incoming connection,
- b is the bias,
- σ is the activation function.

Figure 1.2 shows a schematic view of this basic unit. The set of weights $w_1, \ldots, w_i$ with the bias b constitute the parameters of the neuron, and they are the elements that evolve during training. The magnitude of a weight w (relative to the others) indicates how strongly a neuron depends on the activation of the neurons to which it is connected. A large positive weight means that a high activation in the connected neuron tends to increase the activation of the receiving neuron, whereas a negative weight implies the opposite effect. In this way, the weights modulate the strength and nature of the connections between neurons, thereby defining the overall pattern of interaction within the network.

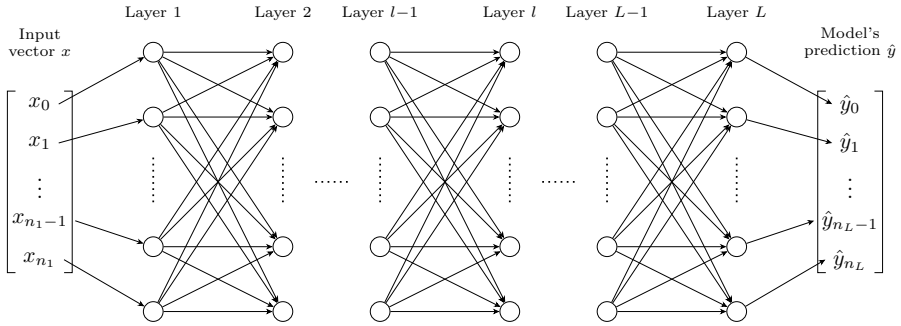


Figure 1.3: Multilayer Perceptron Architecture.

The activation function σ models how a neuron transforms incoming signals into its output activation. Originally, this function was inspired by biological neurons, with the goal of mimicking the firing behavior of organic cells. However, modern DL no longer relies on this biological interpretation. In practice, any function that is nonlinear, differentiable, and computationally efficient can be used. Nonlinearity is a crucial property, since without it a NN would reduce to a composition of linear transformations, and would therefore be unable to approximate complex functions.

1.3.2 Layers

In MLP architectures, neurons are organized into successive layers, as illustrated in Figure 1.3. This layered structure determines how neurons are connected. In this setting, each neuron receives inputs from all neurons in the preceding layer and sends its output to all neurons in the following layer. Consequently, neurons within the same layer do not interact with each other.

Three types of layers are distinguished:

- an input layer, which directly receives input,
- one or more hidden layers, which perform intermediate transformations,
- an output layer, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer: its neurons do not compute activations but instead directly encode the components of the input vector x. Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{y}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NN, especially in MLP, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [Krizhevsky et al., 2012]. Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations

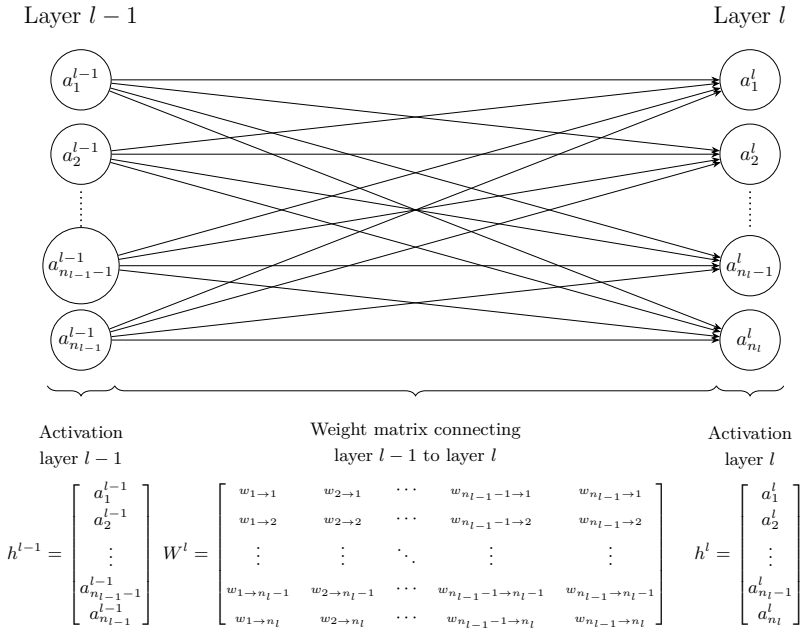


Figure 1.4: Interaction between layers.

of all its neurons, organized into a vector. For a layer l composed of n_l neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [a_1^{(l)}, \ldots, a_{n_l}^{(l)}].$$

Using this formulation, the interaction between two consecutive layers can be written compactly as:

$$\mathbf{h}^{(l)} = \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l-1$ to layer l,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer l,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer l.

This matrix-vector representation is visualised in Figure 1.4, which highlights how each neuron in layer l receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions. In practice, however, modern architectures often include many more. This depth is mainly motivated by empirical observations. Each layer is seen as a processing

step, and more complex target functions typically require more such steps to be well approximated. However, this interpretation is based on experimental findings rather than a theoretical justification [Goodfellow et al., 2016]. An entire chapter of this manuscript is devoted to studying the information that can be extracted from the activations of these intermediate layers.

1.3.3 Network

With a layer-wise representation, the propagation of information can be described as a sequence of transformations applied from the input layer (indexed 1) to the output layer (indexed L):

$$\hat{f}(x) = \mathbf{h}^{(L)} = \sigma^{(L)}\left(\mathbf{W}^{(L)}\sigma^{(L-1)}\left(\dots\sigma^{(1)}\left(\mathbf{W}^{(1)}x + \mathbf{b}^{(1)}\right)\dots\right) + \mathbf{b}^{(L)}\right) = \hat{y}.$$

For a more compact description of information propagation, the layer activations are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} x & \text{if } l = 1, \\ \sigma^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) & \text{if } 2 \leq l \leq L. \end{cases}$$

To simplify notation, it is convenient to group all parameters of the network into a single set. The model parameters are thus denoted by:

$$\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}.$$

In this manuscript, we consider θ as a vector in $\mathbb{R}^P$, where P denotes the total number of parameters $\theta = [w_1, \dots, w_P]$. The notation $\hat{f}_\theta$ expresses the dependence of the NN on its parameters. Increasing the number of parameters, either by adding more neurons per layer or by increasing the number of layers, enhances the expressive power of the model, allowing it to approximate more complex functions. However, this also increases computational cost and training time.

It is important to distinguish between the architecture and the parameters of a NN. When we refer to the architecture of $\hat{f}$, we mean the number of layers, the number of neurons per layer, and the choice of activation functions. The parameters correspond to θ, which controls the strength of the connections between neurons.

1.4 Learning

Now that we have detailed the structure of a NN, we must turn our attention to how it is modified in order to accomplish the task assigned to it. To do so, we need to focus on what lies at the very core of DL: the learning process itself. As a reminder, the objective of a NN is to adjust its parameters so that it becomes a good approximation of a target function. That is, for a given architecture $\hat{f}$, the goal is to find the set of parameters θ that best approximates the target function f. Mathematically, this objective is expressed as:

$$\min_\theta \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}).$$

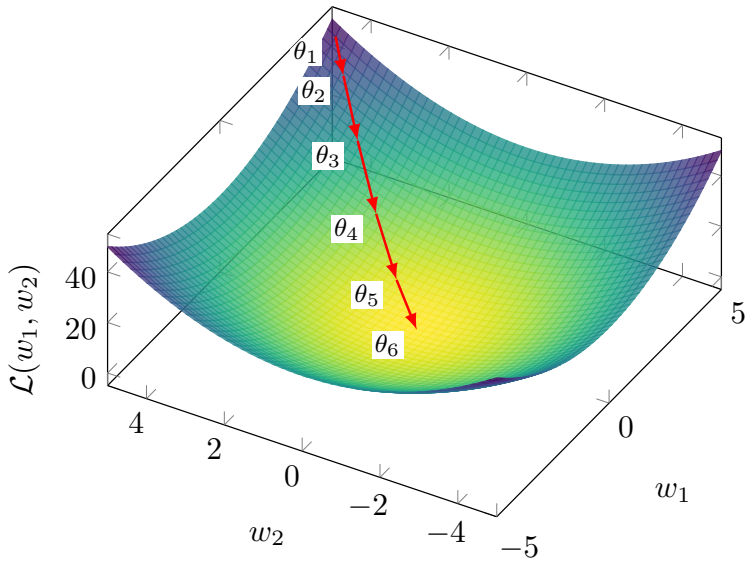


Figure 1.5: Gradient descent in a case $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$.

However, the NN $\hat{f}$ used to approximate the target function f is highly complex. It involves a large number of parameters as well as many nonlinear components. As a result, we cannot find an exact solution by directly studying the properties of the function. So, one must therefore rely on approximate methods to optimize NNs.

The class of algorithms used for this optimization is known as gradient descent methods. To develop an intuition, Figure 1.5 illustrates gradient descent applied to the simple convex loss function $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$, where the red path shows the parameter updates converging to the minimum.

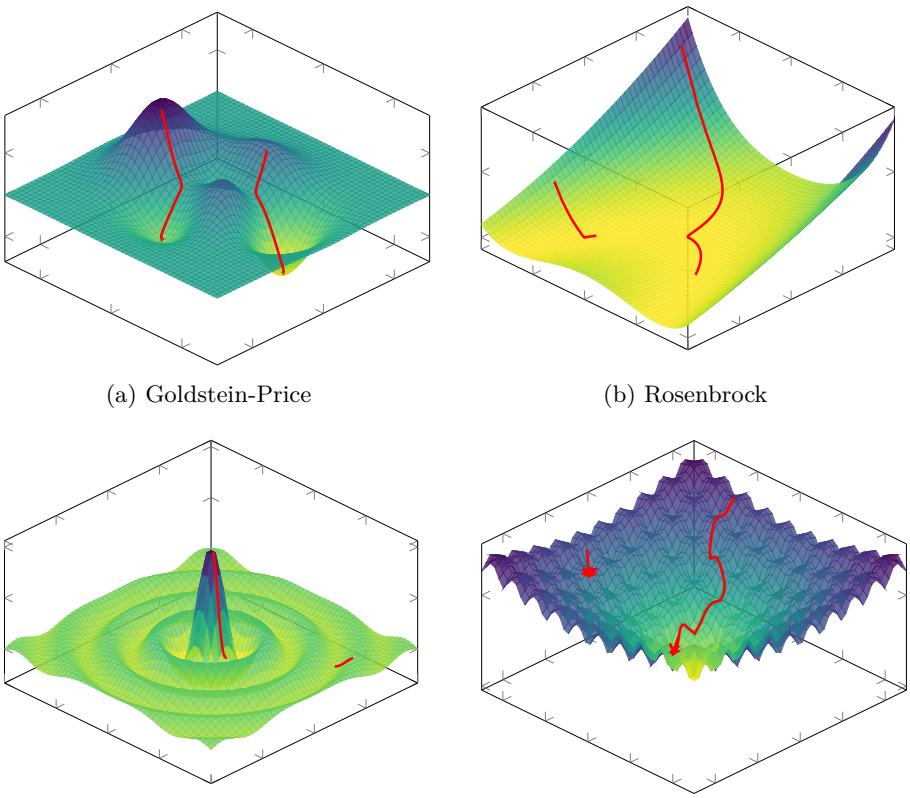
For simplicity, we focus here on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

The first step of SGD is to randomly initialize the parameter vector $\theta \in \mathbb{R}^P$ for a given network architecture $\hat{f}$. We denote this initial state by $\theta^{(1)}$. The goal is then to iteratively modify this vector in order to improve the network's ability to approximate the target function.

To determine how to update the parameters, we study the effect of small variations in each parameter on the loss $\mathcal{L}$. To analyze how parameter variations affect the loss, we rely on a fundamental mathematical tool known as the gradient. It is defined as the collection of partial derivatives of a function with respect to its variables. In our setting, the objective is to understand, for a given example $(x^{(i)}, y^{(i)})$, how the parameters should be adjusted so that the NN prediction $\hat{y}^{(i)}$ becomes closer to the true output $y^{(i)}$. To achieve this, we compute the partial derivatives of the loss function $\mathcal{L}$ with respect to each parameter vector θ:

$$\nabla_\theta \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}) = \left[\frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_1}, \ldots, \frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_P} \right].$$

In neural network training, the loss surface is highly non-convex, meaning



(a) Goldstein-Price (b) Rosenbrock

(c) Sombrero (d) Ackley

Figure 1.6: Comparison of loss surfaces and gradient descent trajectories.

that the optimization path depends strongly on the initialisation. Figure 1.6 illustrates this behaviour for several well-known non-convex functions, showing how different starting points can lead to distinct local minima.

Since both the NN and the loss function are compositions of differentiable functions, these derivatives can be efficiently computed using the chain rule. Each component of the gradient $\frac{\partial \mathcal{L}(\dots)}{\partial w_k}$ $\forall k \in [1, \dots, P]$ indicates how a small increase in the corresponding parameter w_k affects the loss. More precisely, for a given parameter, a positive value in the corresponding gradient component indicates that increasing this parameter will locally increase the loss. Conversely, a negative value indicates that increasing it will locally decrease the loss.

Now that we have information on how changes in the parameters affect the loss, we can aim to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, we move against the gradient because it indicates the direction in which the loss increases, whereas our goal is to reduce it. This leads us to the gradient descent update rule:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_{\theta^{(n)}} \mathcal{L}(\hat{f}_{\theta^{(n)}}(x^{(i)}), y^{(i)}),$$

where $\eta > 0$ is the learning rate, which controls the step size. This parameter must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient

to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\theta_1, \theta_2, \ldots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss.

It is through this simple set of rules: random initialization, gradient computation, and iterative parameter updates, we are able to train highly complex NNs. Of course, when using gradient descent to optimize the parameters of a non-convex function such as a NN, there is no theoretical guarantee of convergence to a global, or even a local, minimum. However, in practice, this method often yields satisfactory results for the task. This success is often attributed to the high expressive power of NNs, even though a complete theoretical understanding of this phenomenon remains an open question.

1.5 Evaluation

Now that we have described how to train a model, we must turn to the question of how to evaluate it. In the supervised learning setting, this evaluation relies on two aspects:

- its performance on the data it was trained on,
- its performance on data it was not trained on.

The first aspect measures how well the loss minimization process has succeeded. The second measures how well the model generalizes. Indeed, the dataset $\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, we study the average loss on the training set, denoted $\bar{\ell}_{\text{train}}$:

$$\bar{\ell}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{train}}} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})$$

If $\bar{\ell}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what we have discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate, or the initialization strategy. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically,

based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. We must now study its ability to generalize. To do so, we compute the corresponding average loss on $\mathcal{D}_{\text{test}}$:

$$\bar{\ell}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{test}}} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})$$

If $\bar{\ell}_{\text{test}} \gg \bar{\ell}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\ell}_{\text{test}} \approx \bar{\ell}_{\text{train}}$, this indicates that the model generalizes well beyond the training data. The model can therefore be used with confidence for the task on which it was trained.

The ability of a neural network to generalize provides evidence that the model has learned more than a simple memorization of the training examples. Instead, it must construct internal representations that capture regularities present in the data. These representations emerge progressively through the hidden layers during training. Understanding their structure is therefore a key step toward understanding how neural networks generalize. The collection of these representations is referred as the latent space. The next chapter is devoted to the study of these latent spaces.

Chapter 2

Latent Space Analysis

The ability of a neural network to generalize provides evidence that the model has learned more than a simple memorization of the training examples. Instead, it must construct internal representations that capture regularities present in the data. These representations play a central role in the network's ability to solve a task. Understanding their structure is therefore a key step toward understanding how neural networks achieve generalization.

In classical machine learning approaches, practitioners had to manually design data representations in order to make them usable by simple models. Deep learning introduces a paradigm shift: rather than relying on hand-crafted features to meaningfully represent data, it automatically learns these representations. Through the successive activation of hidden layers, neural networks gradually construct representations that become increasingly relevant to the task under consideration. As discussed in Chapter 1, these representations are not meaningful at initialization, they emerge progressively during training as the model optimizes its objective. It is precisely through these learned representations that a trained network is able to map an input to an output, including for examples never encountered during training.

The study of these representations has therefore become a central topic in deep learning research, particularly in understanding how information is organized within them, what concepts they encode, and how they influence model behavior. In this chapter, we first make explicit the working hypothesis that underlies most research on latent space analysis. In this manuscript, we refer to this assumption as the "concept hypothesis." To do so, we introduce the notions of abstraction, concept, latent space, and semantics. Once the notions of this conceptual framework has been established, we review the main methods that aim to identify and extract concepts from latent representations.

2.1 Concept Hypothesis

In this section, we introduce the main working hypothesis underlying much of the literature on latent space analysis. Although rarely stated explicitly, this hypothesis is implicitly present in the recurring use of notions such as abstraction, representation, latent space, concept, and semantics. The goal of this section is to make this conceptual framework explicit by formally defining its main

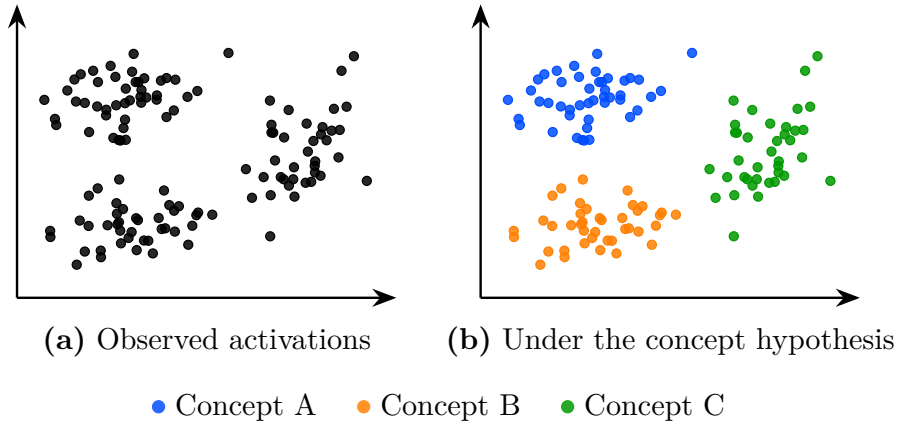


Figure 2.1: The same latent space activations seen without concept hypothesis **(a)** and under the working hypothesis that concepts emerge in the latent space **(b)**. Colors represent hypothetical concept memberships assigned by the model; they are not observed labels. Under this hypothesis the structure of the latent space becomes meaningful and amenable to analysis.

components.

This hypothesis originates from the need to explain the generalization ability of deep neural networks observed after training, as discussed in Section 1.5. Formally, a model is said to generalize when, for an input $x \in \mathcal{X}$ not seen during training, it still produces an output $\hat{y}$ close to the true output y. Such behavior cannot be explained by simple memorization of training samples. Instead, the model must extract and exploit underlying regularities in the target function. This requires a process of abstraction, as defined in Definition 2.1.1, whereby raw inputs are transformed into internal structures that are meaningful for the task.

Definition 2.1.1: Abstraction

Abstraction is the process by which a model transforms an input into a task-relevant internal form. It captures and restructures information in order to represent relationships that are useful for prediction or decision-making.

Through this abstraction process, a neural network does not directly map x to y. Instead, as illustrated in Figure 1.3, it computes a sequence of intermediate activations across hidden layers. These activations correspond to internal encodings of the input, which we refer to as representations in the sense of Definition 2.1.2.

Definition 2.1.2: Representation

A representation is the internal encoding of an input produced by a trained neural network. It corresponds to the activations of one or several layers and provides a transformed view of the input.

Each hidden layer refines the representation of the input. It progressively

transforms it into a structure more directly aligned with the output space. This process can be described as a sequence of transformations where each $h^{(l)}$ is a representation:

$$x \to h^{(1)} \to h^{(2)} \to \cdots \to h^{(L-1)} \to \hat{y}$$

The set of all such representations produced by a layer or a model over a dataset defines its latent space, as defined in Definition 2.1.3.

Definition 2.1.3: Latent Space

The latent space is the set of all representations produced by a layer or model over a dataset. It is shaped by learning and organizes representations according to task-relevant properties.

The existence of meaningful structure in the latent space is closely related to generalization. Under the concept hypothesis, we assume that the neural network internally forms task-dependent abstractions, hereafter referred to as concepts and formally defined in Definition 2.1.4. These concepts are not directly observable, but are expected to manifest in the latent space as recurring and structured patterns in the learned representations. Under this view, a representation can be interpreted as a composition of multiple concepts, each contributing to the model's prediction.

Definition 2.1.4: Concept

A concept is a task-dependent abstraction that captures regularities in the input space relevant for predicting the target output.

Another property that can be considered when analyzing concepts is their relationship to human interpretation. This relationship is captured by the notion of semantics, defined in Definition 2.1.5. Under this definition, concepts may exhibit varying degrees of semantic content. Some concepts can be readily associated with human-interpretable notions, whereas others may contribute to task performance while remaining difficult to interpret.

Definition 2.1.5: Semantics

Semantics refers to the alignment between concepts in the latent space and concepts used by a human observer. A structure is semantic if it can be consistently associated with a human-interpretable concept.

It is important to note that this conceptual framework is not yet supported by a complete theoretical understanding of deep neural networks. Current mathematical tools do not provide a rigorous characterization of their internal representations and do not formally prove that concepts emerge as identifiable structures within latent spaces. Nevertheless, this perspective constitutes a widely adopted working hypothesis in the field of representation and latent space analysis.

The remainder of this chapter is devoted to reviewing methods for analyzing latent spaces and extracting the concepts they encode.

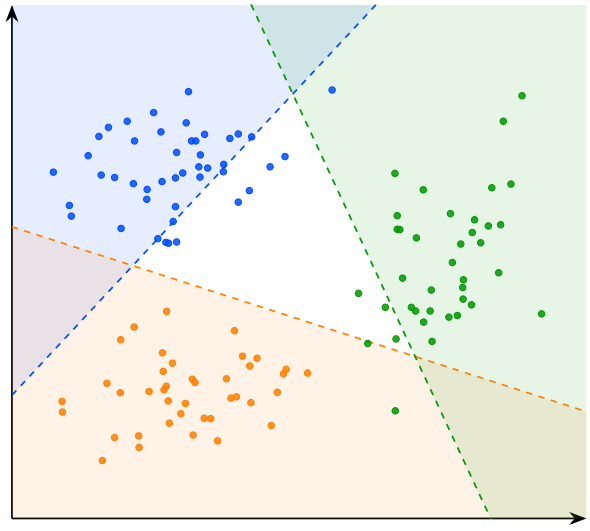


Figure 2.2: Decision boundaries of the linear probes for concepts A, B, and C. The colored regions indicate the areas classified positively by each probe.

2.2 Probing

Probing methods aim to determine whether a given human-defined concept is encoded within the internal representations of a neural network. More specifically, they investigate whether the presence or absence of a selected concept can be inferred from embeddings. The underlying assumption is that if a concept can be reliably decoded from these representations, then the network likely encodes information related to it in order to perform its task.

A probing analysis begins by identifying a set of interpretable concepts that the network could potentially use to accomplish its task. Each example in the dataset is then annotated to indicate the presence or absence of the studied concepts. We denote by c_j the presence or absence of concept j in example x. The dataset can therefore be written as follows:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)}, c_1^{(i)}, \ldots, c_n^{(i)})\}_{i=1}^{N}.$$

The data are subsequently projected into the latent space of the pretrained model. We denote by h^l the representation vector obtained at layer l. From these latent representations, a classifier is trained in a supervised manner to predict the presence of the considered concept. This classifier is referred to as a probe, denoted p, since its role is to determine whether a concept is present or not. Its purpose is therefore to assess whether information associated with the concept is accessible from the network's internal representations. This can be formalized as follows:

$$p_\theta^{l,j} : h^l \mapsto c_j, \quad \theta^* = \arg\min_\theta \mathcal{L}\big(p_\theta^{l,j}(h^l), c_j\big)$$

The classifier used to detect the presence or absence of a concept must remain intentionally simple. Indeed, if the probe has too much capacity, it may reconstruct the target information from complex correlations that do not

necessarily correspond to information explicitly encoded in the latent space. In such a case, it becomes difficult to determine whether the concept is genuinely represented by the network or merely reconstructed through the expressive power of the classifier. For this reason, probing methods often rely on linear classifiers. A linear separation suggests that the information is easily accessible to the neural network and potentially usable during decision-making. If the classifier achieves performance significantly above chance level, this suggests that the latent space contains exploitable information related to the considered concept.

An application of this technique can be found in studies investigating whether large language models (LLMs) develop internal representations of their environment [Li et al., 2024, Karvonen, 2024]. In these works, researchers train LLMs to play games such as chess or Othello in a purely textual setting. The model is only provided with sequences of moves and information about the actions it can take, all expressed in natural language. The central question is whether the LLM develops an internal representation of the game state. To address this question, the probed concepts correspond to the presence or absence of different game pieces on specific board positions. What these studies observe is that probes are able to reconstruct the full board state from the model's hidden representations, even though the only available information to the model is a sequence of text.

However, the fact that a concept can be decoded from latent representations does not necessarily imply that the model actually uses this information to perform its task. To assess whether representations play a causal role in the model's output, it is possible to perform interventions on the latent representations. An intervention consists of modifying a latent representation in a controlled manner by activating or deactivating a given concept, and then studying its impact on the model's output. If the resulting change in the model's output is consistent with the applied perturbation, this suggests that the representation had a causal role.

To this end, we aim to minimally modify the latent representation h^l such that the probe prediction for the target concept c_j changes state, while leaving the predictions associated with other concepts unchanged. This leads to an optimization problem that can be formulated as follows. A common approach to solve this problem is to perform gradient-based optimization on the latent activation h^l, in order to directly optimize the target objective:

$$h^{l*} = \arg\min_{h^{l'}} \mathcal{L}_{\text{flip}}\big(p_\theta^{l,j}(h^{l'}), c_j\big) + \lambda \sum_{k \neq j} \mathcal{L}_{\text{preserve}}\big(p_\theta^{l,k}(h^{l'}), c_k\big) + \beta \|h^{l'} - h^l\|_2^2$$

where:

- $\mathcal{L}_{\text{flip}}$: loss encouraging the prediction of the target concept c_j to change state (i.e., flipping the predicted label),
- $\mathcal{L}_{\text{preserve}}$: loss enforcing invariance of all non-target concepts c_k for $k \neq j$,
- $\|h^{l'} - h^l\|_2^2$: regularization term ensuring the modified representation remains close to the original latent representation,
- λ, β: weighting coefficients controlling the trade-off between preservation, intervention strength, and minimality of the perturbation.

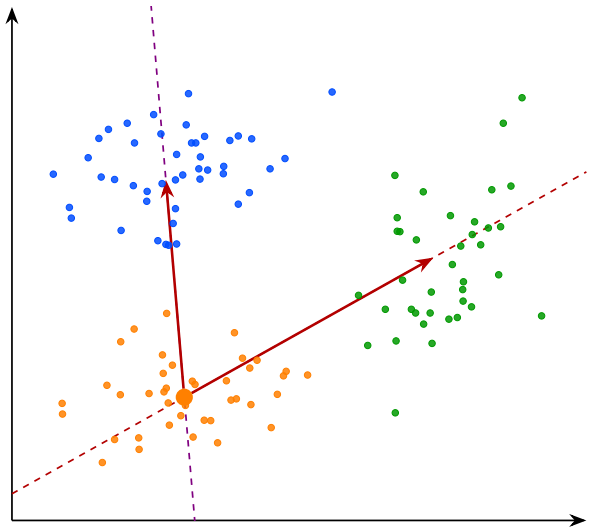


Figure 2.3: Interpretable directions in the latent space. The arrows represent semantic shifts from concept B toward concepts A and C. Moving along these directions in the latent space induces a coherent change in the model's behavior, suggesting that the transition between concepts is continuously encoded along these axes.

In the context of LLMs studied previously, such interventions have been used to remove specific pieces from the model's internal representation of the game state [Li et al., 2024, Karvonen, 2024]. The main observation is that, in the vast majority of cases, the model behaves as if the piece had effectively disappeared. In other words, the LLM continues to generate legal moves while correctly accounting for the absence of the removed piece. This provides evidence for the causal role of these internal representations in the model's behavior.

A main limitation of the probing approach is that the concepts under investigation must be predefined before any analysis can be performed. This introduces a strong bias in what can be studied. Moreover, annotating a dataset for each concept is extremely time-consuming and requires significant human effort. Interventions in latent space are also difficult to interpret, as it is not always clear what constitutes a meaningful change in the network's output under a given perturbation.

Probing methods provide a binary view of concepts, focusing on whether a given concept is present or absent in a latent representation. However, this binary perspective can be overly restrictive, as concepts may be represented in a more continuous or graded manner, with varying degrees of presence. To obtain a more detailed characterization of these representations, other approaches have been proposed, such as the analysis of interpretable directions in latent spaces.

2.3 Interpretable Directions

To overcome the limitations of binary concept detection in probing methods, several approaches have been proposed to analyze the geometric structure of

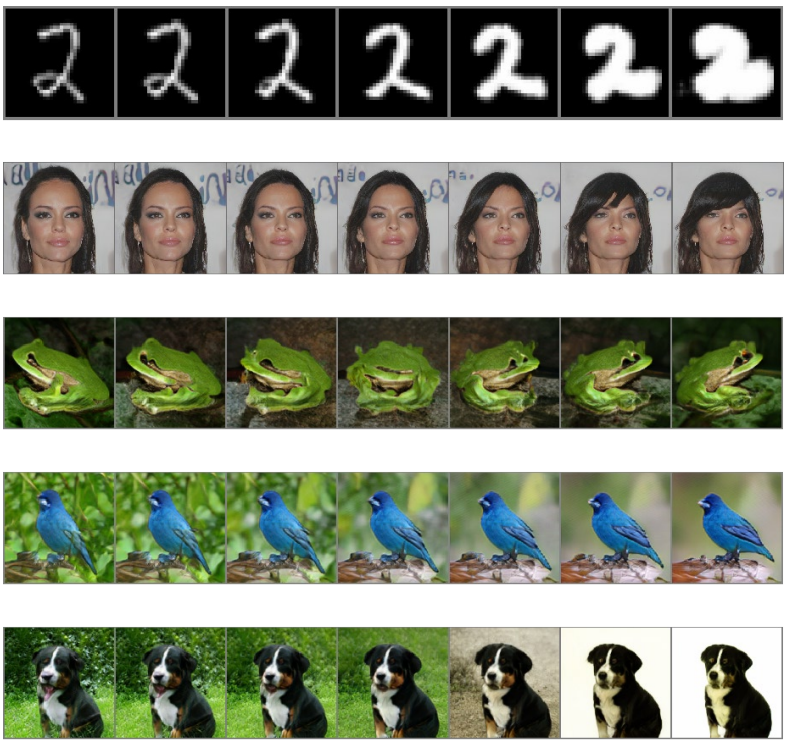


Figure 2.4: Unsupervised discovery of interpretable directions in the latent space of a generative model [Voynov and Babenko, 2020]. Each row corresponds to a distinct direction $\mathbf{v}_i$, and each column to an increasing value of α. The resulting transformations are semantically coherent and diverse: stroke thickness for digits, head rotation for faces, texture and background for natural images.

latent representations. Among them, the notion of interpretable directions has emerged as a natural extension, aiming to capture the continuous nature of how concepts are encoded in representation spaces. Empirically, it has been observed that shifting a latent representation along a direction can induce coherent and semantically meaningful changes in the model's output. Such transformations can be expressed as:

$$h' = h + \alpha \mathbf{v}_i,$$

where:

- h denotes the original embedding,
- $\mathbf{v}_i$ is a unit vector associated with concept c_i,
- α controls the strength of the intervention,
- h' is the resulting perturbed representation.

In many cases, the resulting change in the model's output varies smoothly with α, suggesting an approximately linear structure of the latent space with

respect to certain semantic factors. This observation has led to the hypothesis that neural representations may organize concepts in a partially linear manner within the embedding space. Vectors of this form are referred to as interpretable directions, as they correspond to transformations that can be associated with specific semantic attributes. The objective of this line of work is therefore to identify directions in latent space that align with meaningful variations in the encoded concepts.

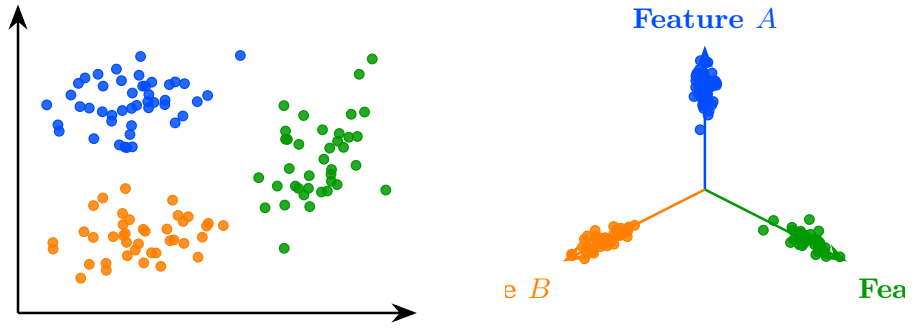
This phenomenon has been extensively studied in generative image models. As illustrated in Figure 2.4, controlled perturbations of latent representations along interpretable directions can induce specific semantic transformations, such as stroke thickness modification in digit images, head rotation in facial images, or texture and background changes in natural images [Voynov and Babenko, 2020]. These observations have led to the development of the field of latent space editing, whose objective is to manipulate generated content directly through modifications in latent space rather than through changes in the input prompt.

However, a central challenge remains: identifying such interpretable directions is far from trivial. Most existing approaches rely on auxiliary models that encode human-defined semantics, such as CLIP, to relate latent variations to textual concepts. Consequently, these methods inherit a limitation similar to that of probing techniques, since concept discovery depends on a pre-existing semantic framework defined by humans.

To overcome this limitation, a research direction known as unsupervised semantic discovery has emerged. Its goal is to uncover interpretable directions in latent spaces without explicit semantic supervision. In these approaches, one model applies perturbations along random directions in latent space, while a second model attempts to predict or recognize the applied transformation. If a stable correspondence emerges between the perturbation and its prediction, this suggests that the considered direction corresponds to a coherent structure in latent space.

In this framework, human intervention occurs only a posteriori, to verify whether the discovered directions indeed correspond to interpretable semantic variations. This approach enables the capture of representations that go beyond the mere presence or absence of a concept, including notions such as the intensity or degree of an attribute. As such, it follows a philosophy similar to causal probing, while also enabling explicit interventions to assess the causal role of latent representations.

However, despite these advances, several challenges remain. First, not all concepts encoded by neural networks appear to be linearly separable in latent space. While some directions align well with clear semantic factors, many others do not exhibit such a simple linear structure. In addition, latent representations often display a phenomenon known as entanglement, where multiple semantic attributes are mixed within the same directions of the representation space. Ideally, one would like each semantic factor to vary independently along a specific direction, without affecting other attributes. In practice, this is rarely the case. A single direction in latent space often influences multiple semantic features at once, meaning that modifying one aspect of the representation can unintentionally alter several others. This lack of separability indicates that semantic information is not cleanly disentangled in the representation space. Together with the difficulty of isolating independent factors, this observation has motivated a stronger hypothesis about how neural networks organize information:



(a) Dense latent space **(b)** Sparse SAE space

Figure 2.5: Transformation from a dense latent space to a sparse SAE-style representation. On the left **(a)**, embeddings are entangled in the initial latent space. On the right **(b)**, the sparse representation aligns each concept with a dedicated feature axis.

the superposition hypothesis.

2.4 Sparse Autoencoder

As discussed previously, approaches based on latent directions face inherent limitations. Empirically, only a restricted subset of concepts appears to admit a linear representation, and even these directions are often affected by significant entanglement with other concepts.

To account for this behavior, a complementary hypothesis has been introduced, extending the assumptions underlying the Concept Hypothesis. This hypothesis, known as the Superposition Hypothesis, posits that semantic concepts are preferentially represented in a linear fashion, with each concept ideally corresponding to a distinct direction in latent space. In this idealized setting, semantic attributes would form an approximately orthogonal basis, where each direction independently controls a single factor of variation. Furthermore, only a small subset of the available concepts is expected to be active for a given representation. As a result, latent representations can be viewed as sparse combinations of concept directions. This idealized representation can be expressed as follows:

$$\mathbf{h} = \sum_{i=1}^{m} a_i \mathbf{v}_i, \qquad \forall i \neq j, \quad \mathbf{v}_i^\top \mathbf{v}_j \approx 0, \qquad \|\mathbf{a}\|_0 \ll m,$$

where:

- $\mathbf{h}$ denotes a latent representation,
- $\mathbf{v}_i$ represents the direction associated with concept c_i,
- a_i corresponds to the activation strength of concept c_i in the representation $\mathbf{h}$,

- $\|\mathbf{a}\|_0 \ll m$ indicates that only a small fraction of concepts are active for a given representation.

According to the Superposition Hypothesis, the reason why latent representations do not perfectly follow this idealized structure is that the representational capacity of neural networks is limited. For a given layer, the dimensionality of the representation space is finite, which constrains the number of concepts that can be encoded independently. As a result, the network cannot allocate a dedicated direction to every concept. Instead, multiple semantic concepts are compressed into shared dimensions, a phenomenon referred to as superposition. Under this hypothesis, a single latent direction may therefore participate in the representation of several distinct concepts. This notion of superposition is closely related to the phenomenon of entanglement observed in latent space analyses. From the perspective both phenomena manifest themselves through the difficulty of isolating semantic factors into independent directions. However, while entanglement is an empirical observation about the structure of latent representations, superposition provides a theoretical explanation for why such mixing may emerge. More generally, superposition can be viewed as a consequence of compression pressure: given a limited representational budget, the model favors efficient encoding of information over a perfectly disentangled organization of concepts.

If this hypothesis holds, then any attempt to analyze latent representations will inevitably be affected by the phenomenon of superposition. Motivated by this view, with the goal of recovering more structured representations a large body of work has focused on developing methods to disentangle latent spaces. Pour ce faire A common strategy consists in projecting a given representation h into a higher-dimensional space, whit additional constraints designed to improve interpretability. The idea is to introduce a projection mechanism that encourages the decomposition of dense representations into a set of simpler components capable of capturing semantic concepts. These approaches are closely related to the field of dictionary learning, which aims to represent complex signals as sparse combinations of elementary features. A widely used implementation of this idea is the sparse autoencoder, which can be defined as follows:

$$s = f_{\text{enc}}(h), \quad \hat{h} = f_{\text{dec}}(s)$$

$$\mathcal{L} = \|h - \hat{h}\|_2^2 + \lambda \|s\|_1$$

where:

- $f_{\text{enc}}(h)$ is the encoding function that projects the original representation space into a sparse latent space,
- $f_{\text{dec}}(s)$ is the decoding function that reconstructs the original representation from the sparse code,
- s is the sparse representation of h,
- $\|h - \hat{h}\|_2^2$ is the reconstruction loss ensuring that the original representation can be faithfully recovered,
- $\lambda \|s\|_1$ is the sparsity regularization term that encourages most components of s to be zero.

Beyond this proof of concept, the method proves powerful when applied to real language models. In a onelayer transformer, training a sparse autoencoder on MLP activations reveals a set of clean, humaninterpretable features functional causal units that do not correspond to individual neurons. For instance, specific autoencoder features selectively activate on base64encoded text or on Arabic script, and manually activating them causes the model to generate outputs in the corresponding format.

Despite their empirical success, sparse autoencoders rely on strong structural assumptions that are not always justified. First, the learned concepte are not guaranteed to be identifiable: different training runs may yield different decompositions of the same latent space, even when the reconstruction performance is similar. Second, the assumption that the underlying representation is approximately linearly decomposable may not hold uniformly across all regions of the latent space, especially in highly nonlinear regimes. Finally, sparsity itself introduces a modeling bias that may favor certain types of concepte while suppressing others.

These limitations suggest that sparse autoencoders, while powerful, do not fully resolve the problem of entanglement in neural representations. They instead provide one possible coordinate system in which parts of the structure become more explicit, but not necessarily a complete or canonical one.

From this perspective, both interpretable directions and sparse autoencoders rely on a shared underlying assumption: that semantic structure can be recovered through linear or approximately linear decompositions. While this assumption has led to significant empirical progress, it remains theoretically fragile.

The following section therefore introduces a complementary approach that relaxes this constraint and does not require a globally linear feature decomposition of the latent space.

2.5 Clustering

The latent space analysis methods presented so far all rely, either explicitly or implicitly, on assumptions about the linear organization of concepts within the latent space. For probing methods, concepts must be at least partially linearly separable for simple probes to succeed. Interpretability through latent directions assumes that variations associated with a concept can be captured by moving along a linear axis of the latent space. Similarly, the superposition hypothesis assumes that concepts are represented through linear directions that become entangled because of representational capacity constraints. Although these assumptions have led to numerous empirical successes, their theoretical justification remains limited. More importantly, several observations suggest that the linearity hypothesis may not fully capture the structure of latent representations.

For instance, some concepts can only be recovered using non-linear probes. Likewise, methods based on interpretable directions typically identify only a limited number of semantic factors. Finally, in sparse autoencoder approaches, enforcing excessive sparsity often degrades reconstruction quality and interpretability, suggesting that the underlying representation may not be perfectly described by a sparse linear decomposition.

Taken together, these observations indicate that semantic concepts may not

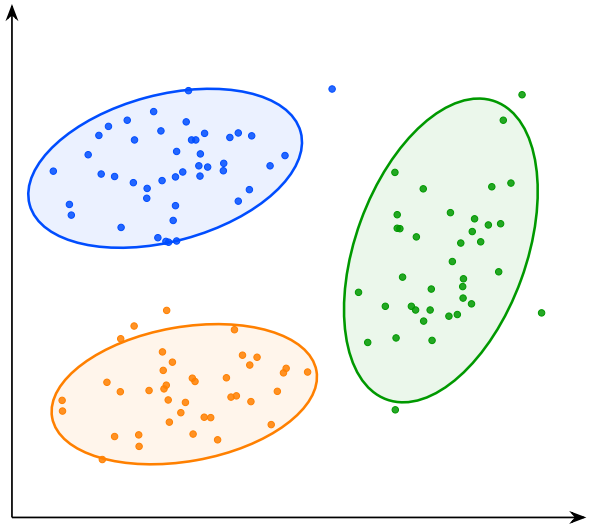


Figure 2.6: Visualisation de la structure de l'espace latent. Les points représentent les embeddings associés aux concepts A, B et C. Les ellipses indiquent l'étendue approximative de chaque groupe dans l'espace latent et mettent en évidence leur séparation géométrique.

always be organized according to a simple linear structure. This motivates the exploration of alternative approaches that rely on weaker assumptions regarding the geometry of latent spaces. One such approach is clustering-based analysis. Rather than assuming that concepts correspond to directions or linear subspaces, clustering methods only assume that semantically similar inputs produce nearby latent representations. Conversely, inputs that differ significantly from a semantic perspective are expected to be located farther apart in the latent space.

Under this assumption, the discovery of concepts can be reformulated as the identification of groups of embeddings that occupy the same region of the latent space. Concepts are therefore no longer associated with directions, but with clusters of neighboring representations. This idea forms the basis of the research field known as deep clustering. In these approaches, a collection of inputs is first projected into a latent space using a neural network. The resulting embeddings form a point cloud on which standard clustering algorithms can be applied. The obtained clusters are then interpreted as representing distinct semantic concepts.

This methodology enables the discovery of semantic structure in the latent space without requiring concepts to be linearly encoded. As a result, it can reveal forms of organization that would remain inaccessible to methods based solely on linear separability or interpretable directions. Deep clustering has been particularly successful in unsupervised image classification. Typically, an autoencoder is trained to learn a compact latent representation that preserves the essential information contained in the input data. Clustering algorithms are then applied to the learned embeddings. Empirically, the resulting clusters often correspond to meaningful semantic categories, despite the absence of supervision during training.

More advanced approaches combine representation learning and clustering objectives within a single optimization process. In this setting, additional loss

terms encourage embeddings to organize themselves around cluster centroids during training. This often produces latent spaces that are easier to analyze and improves the quality of the discovered semantic structure.

Despite its effectiveness, clustering-based analysis also presents important limitations. First, as with most unsupervised semantic discovery techniques, the interpretation of the resulting clusters remains partially subjective. While benchmark datasets often provide class labels that facilitate evaluation, there is generally no guarantee that the discovered clusters correspond to the categories that humans consider most relevant. Alternative semantic groupings may be equally valid.

Second, clustering methods are highly sensitive to design choices. The number of clusters, the distance metric, and the clustering algorithm itself can significantly influence the results. Selecting these parameters is often challenging and may require substantial empirical tuning. Finally, semantic concepts do not necessarily form well-separated clusters in latent space. Some concepts may overlap, exhibit hierarchical relationships, or vary continuously rather than discretely. In such cases, a clustering-based description may provide only a partial view of the underlying representational structure.

Bibliography

[Brown et al., 2020] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901.

[Covington et al., 2016] Covington, P., Adams, J., and Sargin, E. (2016). Deep neural networks for youtube recommendations. In Proceedings of the 10th ACM conference on recommender systems, pages 191–198.

[Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. (2016). Deep learning, volume 1. MIT press Cambridge.

[Hilbert and López, 2011] Hilbert, M. and López, P. (2011). The world's technological capacity to store, communicate, and compute information. science, 332(6025):60–65.

[Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8):1735–1780.

[Hornik et al., 1989] Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. Neural networks, 2(5):359–366.

[Jumper et al., 2021] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. (2021). Highly accurate protein structure prediction with alphafold. nature, 596(7873):583–589.

[Karvonen, 2024] Karvonen, A. (2024). Emergent world models and latent variable estimation in chess-playing language models.

[Krizhevsky et al., 2012] Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25.

[LeCun et al., 1989] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., and Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural computation, 1(4):541–551.

[Li et al., 2024] Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H., and Wattenberg, M. (2024). Emergent world representations: Exploring a sequence model trained on a synthetic task.

[McCulloch and Pitts, 1943] McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5(4):115–133.

[Minsky and Papert, 1969] Minsky, M. and Papert, S. (1969). An introduction to computational geometry. Cambridge tiass., HIT, 479(480):104.

[Mitchell, 1997] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill, New York.

[Raina et al., 2009] Raina, R., Madhavan, A., Ng, A. Y., et al. (2009). Large-scale deep unsupervised learning using graphics processors. In Icml, volume 9, pages 873–880.

[Rosenblatt, 1958] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. Psychological review, 65(6):386.

[Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. nature, 323(6088):533–536.

[Silver et al., 2017] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815.

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.

[Voynov and Babenko, 2020] Voynov, A. and Babenko, A. (2020). Unsupervised discovery of interpretable directions in the gan latent space.